```
**Student Performance Score Predictor**

Import libraries
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.tree import DecisionTreeRegressor

from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

import joblib
import warnings
warnings.filterwarnings("ignore")

""" Load CSV file from Kaggle dataset"""

df = pd.read_csv("/content/student_habits_performance.csv")

print("CSV file loaded successfully")
print("Dataset shape:", df.shape)

df.head()

""" Check dataset information"""

df.info()
```

```
""" Check missing values"""

# Step 9: Check missing values
df.isnull().sum()

"""Showing basic statistics"""

df.describe()

"""Target column"""

target = "exam_score"

print("Target column:", target)
print(df[target].describe())

""" Visualize target column"""

plt.figure(figsize=(8, 5))
sns.histplot(df[target], kde=True, bins=20)
plt.title("Distribution of Student Exam Scores")
plt.xlabel("Exam Score")
plt.ylabel("Count")
plt.show()

"""Correlation heatmap"""

plt.figure(figsize=(10, 6))
sns.heatmap(
    df.select_dtypes(include=["int64", "float64"]).corr(),
    annot=True,
    cmap="coolwarm"
)
plt.title("Correlation Heatmap")
plt.show()

"""Split input features and output target"""

X = df.drop(columns=[target])
y = df[target]
```

```
print("Input features shape:", X.shape)
print("Target shape:", y.shape)

"""Detect numerical and categorical columns"""

numerical_features = X.select_dtypes(include=["int64",
"float64"]).columns.tolist()
categorical_features = X.select_dtypes(include=
["object"]).columns.tolist()

print("Numerical columns:")
print(numerical_features)

print("\nCategorical columns:")
print(categorical_features)

"""Train-test split"""

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training rows:", X_train.shape[0])
print("Testing rows:", X_test.shape[0])

"""Preprocessing for CSV data"""

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_pipeline, numerical_features),
        ("cat", categorical_pipeline, categorical_features)
    ]
)

"""Train multiple machine learning models"""

models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(random_state=42),
    "Gradient Boosting": GradientBoostingRegressor(random_state=42)
}

results = []

for name, model in models.items():
    ml_pipeline = Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("model", model)
    ])

    ml_pipeline.fit(X_train, y_train)
    y_pred = ml_pipeline.predict(X_test)

    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)

    results.append({
        "Model": name,
        "MAE": mae,
        "RMSE": rmse,
        "R2 Score": r2
    })

results_df = pd.DataFrame(results)
results_df = results_df.sort_values(by="R2 Score", ascending=False)
```

```
results_df

"""linear regression"""

plt.figure(figsize=(10, 6))
sns.barplot(x="Model", y="R2 Score", data=results_df, palette="viridis")
plt.title("Model Performance Comparison (R2 Score)")
plt.xlabel("Model")
plt.ylabel("R2 Score")
plt.ylim(0, 1) # R2 score typically ranges from 0 to 1
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

"""Select best model automatically"""

best_model_name = results_df.iloc[0]["Model"]

print("Best model:", best_model_name)

""" Train final model using Random Forest"""

final_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", RandomForestRegressor(
        n_estimators=200,
        random_state=42
    ))
])

final_model.fit(X_train, y_train)

final_predictions = final_model.predict(X_test)

print("Final Model Evaluation")
print("MAE:", mean_absolute_error(y_test, final_predictions))
print("RMSE:", np.sqrt(mean_squared_error(y_test, final_predictions)))
print("R2 Score:", r2_score(y_test, final_predictions))

""" Compare actual and predicted values"""
```

```
comparison_df = pd.DataFrame({
    "Actual Exam Score": y_test.values,
    "Predicted Exam Score": final_predictions
})

comparison_df.head(20)

"""Actual vs predicted graph"""

plt.figure(figsize=(7, 5))
sns.scatterplot(x=y_test, y=final_predictions)
plt.xlabel("Actual Exam Score")
plt.ylabel("Predicted Exam Score")
plt.title("Actual vs Predicted Student Exam Scores")
plt.show()

"""Save trained model"""

joblib.dump(final_model, "student_performance_predictor.pkl")

print("Model saved as student_performance_predictor.pkl")

""" Predict score for a new student"""

new_student = pd.DataFrame([
    {
        "student_id": "S_new", # Dummy ID
        "age": 20, # Example value, adjusted from original 23
        "gender": "Male",
        "study_hours_per_day": 3.5, # Adjusted from 22 to a realistic
range based on df.describe()
        "social_media_hours": 2.5, # Example value
        "netflix_hours": 1.8, # Example value
        "part_time_job": "No", # Example value
        "attendance_percentage": 88,
        "sleep_hours": 7,
        "diet_quality": "Good", # Example value
        "exercise_frequency": 3,
        "parental_education_level": "Some College", # Mapping "College"
to available categories
        "internet_quality": "Good", # Example value, mapping from
```

```
    "Internet_Access"
            "mental_health_rating": 5, # Mapping "Motivation_Level: Medium"
    to a mental health rating
            "extracurricular_participation": "Yes"
        }
    ])

    predicted_score = final_model.predict(new_student)

    print("Predicted Student Exam Score:", round(predicted_score[0], 2))

    """Pass / Fail result"""

    if predicted_score[0] >= 50:
        print("Result: Student is likely to PASS")
    else:
        print("Result: Student is likely to FAIL")

    """ Download saved model from Colab"""

    from google.colab import files

    files.download("student_performance_predictor.pkl")

    """CSV file downloaded from Kaggle"""

    df = pd.read_csv("/content/student_habits_performance.csv")
```